

ANÁLISIS DEL RETO

Juan Sebastian rojas Marin, 202610140, js.rojasm123@uniandes.edu.co

Estudiante 2, código 2, email 2

Estudiante 3, código 3, email 3

Requerimiento <<1>>

Descripción

```
42 def req_1(catalog,product):
43     """
44     Implement with Codex
45     # 1000: Modificar el requerimiento 1
46     start_time = get_time()
47     orders = catalog["orders"]
48     totals = {}
49     count=0
50     sum_price=0
51     min_price=None
52     max_price=None
53     sum_discount=0
54     min_discount=None
55     max_discount=None
56     sum_boxes=0
57     min_boxes=None
58     max_boxes=None
59     sum_marketing=0
60     min_marketing=None
61     max_marketing=None
62     years={}
63     max_amount_order=None
64     min_amount_order=None
65     for i in range(len(orders)):
66         order= orders[i].get_element(orders,i)
67         if order["Product"]!=product:
68             continue
69         count+=1
70         price=float(order["Price_per_Box"])
71         discount=float(order["Discount_Pct"])
72         boxes=int(order["Boxes Shipped"])
73         marketing=float(order["Marketing_Spend"])
74         amount=float(order["Amount"])
75         year=order["Order Date"][:4]
76         sum_price+=price
77         if min_price is None or price < min_price:
78             min_price=price
79         if max_price is None or price > max_price:
80             max_price=price
81         sum_discount+=discount
82         if min_discount is None or discount < min_discount:
83             min_discount=discount
84         if max_discount is None or discount > max_discount:
85             max_discount=discount
86         sum_boxes+=boxes
87         if min_boxes is None or boxes < min_boxes:
88             min_boxes=boxes
89         if max_boxes is None or boxes > max_boxes:
90             max_boxes=boxes
91         sum_marketing+=marketing
92         if min_marketing is None or marketing < min_marketing:
93             min_marketing=marketing
94         if max_marketing is None or marketing > max_marketing:
95             max_marketing=marketing
96         years[year]=years.get(year,0)+1
97         if max_amount_order is None:
98             max_amount_order= order
99         elif amount>float(max_amount_order["Amount"]):
100             max_amount_order=order
101         elif amount == float(max_amount_order["Amount"]) and marketing < float(max_amount_order["Marketing_Spend"]):
102             max_amount_order=order
103     years[year]=years.get(year,0)+1
104     if max_amount_order is None:
105         max_amount_order= order
106     elif amount>float(max_amount_order["Amount"]):
107         max_amount_order=order
108     elif amount == float(max_amount_order["Amount"]) and marketing < float(max_amount_order["Marketing_Spend"]):
109         max_amount_order=order
110     end_time = get_time()
```

```
sum_price+=price
if min_price is None or price < min_price:
    min_price=price
if max_price is None or price > max_price:
    max_price=price

sum_discount+=discount
if min_discount is None or discount < min_discount:
    min_discount=discount
if max_discount is None or discount > max_discount:
    max_discount=discount

sum_boxes+=boxes
if min_boxes is None or boxes < min_boxes:
    min_boxes=boxes
if max_boxes is None or boxes > max_boxes:
    max_boxes=boxes

sum_marketing+=marketing
if min_marketing is None or marketing < min_marketing:
    min_marketing=marketing
if max_marketing is None or marketing > max_marketing:
    max_marketing=marketing

years[year]=years.get(year,0)+1

if max_amount_order is None:
    max_amount_order= order
elif amount>float(max_amount_order["Amount"]):
    max_amount_order=order
elif amount == float(max_amount_order["Amount"]) and marketing < float(max_amount_order["Marketing_Spend"]):
    max_amount_order=order
```

```
if min_amount_order is None:
    min_amount_order= order
elif amount<float(min_amount_order["Amount"]):
    min_amount_order=order
elif amount == float(min_amount_order["Amount"]) and marketing < float(min_amount_order["Marketing_Spend"]):
    min_amount_order=order

top_year=None
top_year_count=0
for y, c in years.items():
    if c>top_year_count:
        top_year=y
        top_year_count=c
end_time = get_time()
```

```

result = {
    "product": product,
    "count": count,
    "avg_price": sum_price/count if count>0 else 0,
    "min_price": min_price,
    "max_price": max_price,
    "avg_discount": sum_discount/count if count>0 else 0,
    "min_discount": min_discount,
    "max_discount": max_discount,
    "avg_boxes": sum_boxes/count if count>0 else 0,
    "min_boxes": min_boxes,
    "max_boxes": max_boxes,
    "avg_marketing": sum_marketing/count if count>0 else 0,
    "min_marketing": min_marketing,
    "max_marketing": max_marketing,
    "years": years,
    "top_year": top_year,
    "top_year_count": top_year_count,
    "max_amount_order": max_amount_order,
    "min_amount_order": min_amount_order,
}
return result, delta_time(start_time, end_time)

```

Este requerimiento recibe el catálogo de pedidos y el nombre de un producto, y recorre la totalidad de los pedidos para obtener únicamente aquellos que corresponden al producto solicitado. Para cada pedido que cumple el filtro, se aumenta el conteo total, las sumas del producto en el catálogo, los valores mínimo y máximo de precio por caja, porcentaje de descuento, cajas despachadas y gasto en marketing, además de contar la cantidad de pedidos por año en un diccionario, con esto se logra hallar el año con mayor número de pedidos (top_year). También compara cada pedido para conservar el de mayor y el de menor monto (Amount), usando el gasto en marketing como criterio para que no exista un error al tener un “empate”. Finalmente calcula los promedios acumulados y construye el diccionario de resultado con todas las estadísticas solicitadas.

Entrada	Catalog, product
Salidas	Diccionario return (con toda la información generada por el requerimiento) y Delta_time (Tiempo de ejecución del requerimiento)
Implementado (Sí/No)	Si y se implementó por Juan Sebastian Rojas Marin

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener la lista de pedidos y su tamaño (lt.size)	O(1)
Recorrer todos los pedidos con lt.get_element	O(n)
Filtrar por producto y actualizar sumatorias, mínimos y máximos (precio, descuento, cajas, marketing)	O(1) por pedido

Actualizar el conteo de pedidos por año en el diccionario years	O(1) por pedido
Comparar y actualizar el pedido de mayor y de menor monto	O(1) por pedido
Recorrer el diccionario years para hallar el año con más pedidos (top_year)	O(n)
Calcular promedios y construir el diccionario de resultado	O(1)
TOTAL	O(n)

Análisis

El requerimiento 1 tiene complejidad total $O(n)$, siendo n el número de pedidos del catálogo, ya que al no existir una función auxiliar indexada por producto es obligatorio recorrer todos los pedidos para filtrar los del producto buscado. Las operaciones dentro de ese recorrido (sumas, mínimos, máximos, actualización del diccionario de años y del pedido de mayor/menor monto) son $O(1)$, y el recorrido final sobre el diccionario de años depende del número de años distintos, una cantidad pequeña que no afecta el orden general.

La lista de pedidos se implementó con la estructura de datos de una Array List (lt) porque solo se necesita un recorrido secuencial sin inserciones ni eliminaciones, y con el TAD de `get_element()` en $O(1)$ el recorrido completo queda en $O(n)$; con una lista enlazada y acceso por índice el costo habría sido $O(n^2)$.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):
    """
    Retorna un dato a partir de su ID
    """
    pos_data = lt.isPresent(data_structs["data"], id)
    if pos_data > 0:
        data = lt.getElement(data_structs["data"], pos_data)
        return data
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
----------------	--------------------------------------

Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.